



Guion literal del video — Tarea 3: Pruebas Automatizadas

Duración objetivo: 9–11 minutos · **Participantes:** 5 · **Formato:** grabación de pantalla (presentación PDF + VS Code + terminal) con voz de cada integrante.

Reemplacen [PERSONA 1..5] con sus nombres. Cada quien puede leer su parte tal cual está escrita o parafrasearla — lo importante es no omitir las frases marcadas en **negrita**, porque son las que responden directamente a la rúbrica.

Cobertura de la rúbrica (para que nada quede fuera)

Rubro	Puntos	¿Quién lo cubre?	¿Dónde?
Frameworks existentes (front y back)	20	PERSONA 2	Escena 2 (diapositivas 2–3)
Razones de la elección (comunidad, docs, rendimiento, etc.)	25	PERSONA 3	Escena 3 (diapositivas 4–5)
Ejemplos + video de implementación y funcionamiento	35	PERSONA 4 y PERSONA 5	Escenas 4–5 (diapositivas 6–9 + demo en vivo)
Conclusiones y experiencia	20	PERSONA 1	Escena 6 (diapositivas 10–11)

Preparación ANTES de grabar (checklist)

1. Abrir `docs/tarea3-pruebas-automatizadas/presentacion.pdf` en pantalla completa.
2. Tener VS Code abierto en el proyecto Kontrol con estas pestañas listas:
 - `backend/vitest.config.js` y `frontend/vitest.config.js`
 - `backend/tests/requireAuth.test.js`
 - `backend/tests/validate.test.js`
 - `frontend/tests/authStore.test.js`
 - `frontend/tests/Button.test.js`
 - `frontend/src/components/UI/Button/Button.vue`
3. Dos terminales abiertas: una en `backend/` , otra en `frontend/` .
4. Probar antes de grabar que estos comandos corren en verde:
 - `cd backend && npm test`
 - `cd frontend && npm test`
 - `cd backend && npm run test:coverage`
 - `cd frontend && npm run test:watch` (se cierra con `q`)
5. Micrófono probado; grabar con OBS, la grabadora de pantalla del sistema, o Zoom/Meet grabando la pantalla compartida.

ESCENA 1 — Introducción · PERSONA 1 (~45 segundos)

[PANTALLA: diapositiva 1 — portada]

"Hola, buenos días/tardes. Somos el equipo de Kontrol, integrado por [nombres de los 5]. En este video presentamos nuestra Tarea 3 de Ingeniería de Software 2, sobre pruebas unitarias automatizadas.

Kontrol es nuestra plataforma de gestión empresarial: el **backend** está construido con **Node.js 20 y Express**, con MongoDB y PostgreSQL como bases de datos; y el **frontend** con **Vue 3, Vite y Pinia** para el manejo de estado.

En este video vamos a mostrar: primero, los frameworks de pruebas que investigamos para nuestra tecnología; segundo, por qué elegimos la herramienta que elegimos;

tercero, ejemplos reales de pruebas unitarias funcionando en nuestro proyecto, tanto de backend como de frontend; y por último, nuestras conclusiones."

ESCENA 2 — Frameworks investigados · PERSONA 2 (~2 minutos) — *rubro de 20 pts*

[PANTALLA: diapositiva 2 — frameworks backend]

"Empezamos investigando **qué frameworks existen para automatizar pruebas unitarias en Node.js**, que es la tecnología de nuestro backend. Encontramos cinco opciones principales:

Jest, creado por Meta, es el framework más usado del ecosistema JavaScript, con alrededor de 25 millones de descargas semanales. Trae todo incluido: runner, aserciones, mocks y cobertura. Pero tiene un problema importante para nosotros: su soporte de **módulos ES es experimental**, y nuestro backend está escrito completamente en ESM — en el package.json tenemos `"type": "module"` .

Vitest es el framework moderno del equipo de Vite y Vue. Su API es compatible con la de Jest, soporta ESM de forma **nativa**, y también trae mocking y cobertura integrados.

Mocha con Chai y Sinon es el stack clásico: es muy estable, pero hay que ensamblar tres librerías por separado, y ni la cobertura ni el mocking vienen incluidos.

node:test es el runner nativo de Node 20: no requiere instalar nada, pero su mocking y sus reportes todavía son muy básicos.

Y **AVA**, un runner minimalista, con una comunidad pequeña y sin mocking integrado."

[PANTALLA: diapositiva 3 — frameworks frontend]

"Del lado del frontend, para **Vue 3**, las opciones son:

Vitest con @vue/test-utils, que es la **combinación oficial recomendada por el equipo de Vue** — de hecho es la que instala por defecto el scaffold oficial `create-vue` .

Jest con vue3-jest, que era el estándar en la época de Vue 2 con webpack, pero hoy

exige duplicar configuración con Babel y transformadores, y su mantenimiento está rezagado.

Testing Library, que no es un runner sino una capa encima de test-utils, enfocada en probar como usuario. Es un complemento, no una alternativa.

Y **Cypress o Playwright Component Testing**, que montan componentes en un navegador real: máxima fidelidad, pero un setup pesado y lento — excesivo para pruebas unitarias.

Para simular el DOM en los tests existen **happy-dom**, que es rápido y ligero, y **jsdom**, más completo pero más lento."

ESCENA 3 — Elección y configuración · PERSONA 3 (~2 minutos) — *rubro de 25 pts*

[PANTALLA: diapositiva 4 — elección]

"Con esa investigación, decidimos usar **Vitest como framework único para backend y frontend**. Las razones:

Primero, **integración con nuestro stack**: el frontend ya usa Vite, así que Vitest **reutiliza nuestra configuración de Vite tal cual** — los alias y el plugin de Vue funcionan en los tests sin configurar nada extra. Y como nuestro backend es ESM puro, el soporte nativo de ESM de Vitest nos evita los flags experimentales que necesitaría Jest.

Segundo, **rendimiento**: Vitest transforma el código con esbuild, que está escrito en Go, corre los tests en workers paralelos, y su modo watch funciona como el hot-reload de Vite: al guardar un archivo, **solo re-ejecuta los tests afectados**, con feedback casi instantáneo. Esto lo van a ver en vivo más adelante.

Tercero, **comunidad y documentación**: más de 10 millones de descargas semanales, es el default de Vue, Nuxt, Astro y SvelteKit, tiene documentación completa en vitest.dev y desarrollo muy activo. Además su **API es compatible con Jest**: describe, it, expect, mocks — todo lo que ya conocíamos aplica directamente, así que la curva de aprendizaje fue prácticamente cero.

Y cuarto, **robustez**: trae mocks, spies, fake timers y cobertura integrados, sin librerías adicionales. Un solo framework para todo el monorepo significa los mismos comandos y los mismos reportes en backend y frontend.

Como complementos usamos `@vue/test-utils` para montar componentes, `happy-dom` como DOM simulado y `@vitest/coverage-v8` para cobertura."

[PANTALLA: diapositiva 5 — configuración]

"La configuración fue mínima. En el backend, el archivo completo son **8 líneas**: entorno node y dónde están los tests. En el frontend usamos `mergeConfig` para **heredar toda la configuración de Vite** — por eso el alias arroba y los archivos `.vue` funcionan sin tocar nada — y solo agregamos el entorno happy-dom.

En ambos package.json agregamos tres scripts: `npm test` para correr la suite una vez, `npm run test:watch` para el modo watch, y `npm run test:coverage` para el reporte de cobertura."

ESCENA 4 — Ejemplos backend + demo · PERSONA 4 (~2.5 minutos) — *rubro de 35 pts, parte 1*

[PANTALLA: diapositiva 6 — requireAuth]

"Ahora los ejemplos reales en nuestro proyecto. En el backend escribimos **cuatro suites con 22 pruebas**. Les muestro las más importantes.

La primera prueba nuestro middleware de **autenticación con JWT**, `requireAuth`. La clave está en que un middleware de Express es solo una función que recibe `req`, `res` y `next` — así que no necesitamos levantar servidor ni base de datos: construimos un `req` falso con el header `Authorization`, simulamos `res.status` y `res.json` con `vi.fn()`, que es la función de mocking de Vitest, y firmamos **tokens JWT reales** con el secreto de prueba.

Cubrimos cinco escenarios: token válido — donde verificamos que se llama `next()` y

que el payload queda en `req.user` —, petición sin header, header que no es Bearer, **token expirado** — que generamos firmando con expiración negativa — y token firmado con otro secreto. Los cuatro últimos deben responder 401."

[PANTALLA: diapositiva 7 — `validate` y `requireRole`]

"La segunda suite prueba nuestro middleware de **validación con Zod**. Aquí hay un detalle interesante: verificamos no solo que un body inválido responde 400 con la lista de errores por campo, sino que un body válido **reemplaza `req.body` con los datos ya parseados** — por ejemplo, la edad llega como texto '25' y el controller la recibe como número 25.

La tercera prueba el middleware de **autorización por roles**: un rol permitido pasa, un rol no permitido recibe 403, y una petición sin usuario autenticado también recibe 403.

Y la cuarta suite prueba las utilidades del chat: sanitización de archivos adjuntos, generación de previews y serialización de mensajes — 10 pruebas de funciones puras."

[PANTALLA: cambiar a la terminal en `backend/`]

Ejecutar en vivo:

```
npm test
```

"Y aquí está corriendo en vivo: **22 pruebas, 4 archivos, todas en verde, en menos de medio segundo.**"

Ejecutar en vivo:

```
npm run test:coverage
```

"Con un solo comando extra tenemos el reporte de cobertura: en los archivos que probamos alcanzamos alrededor de **80 % de statements**, con el detalle línea por línea de lo que falta cubrir."

ESCENA 5 — Ejemplos frontend + demo del bug + watch · PERSONA 5 (~2.5 minutos) — *rubro de 35 pts, parte 2*

[PANTALLA: diapositiva 8 — store Pinia]

"En el frontend escribimos otras **cuatro suites con 31 pruebas**.

La más completa prueba nuestro **store de autenticación de Pinia**, que es el corazón de la sesión en Kontrol. Con `setActivePinia` creamos un store limpio para cada test, y con `vi.stubGlobal` **reemplazamos fetch por un mock**, así probamos la lógica sin tocar la API real. Por ejemplo: verificamos que `loadEmpresas` llama al endpoint correcto con el token en el header, y que **auto-selecciona la primera empresa** cuando el usuario no tenía ninguna seleccionada. También probamos login, logout, la persistencia en `localStorage` — que happy-dom nos da de verdad — y los getters de permisos."

[PANTALLA: diapositiva 9 — Button y statusHelpers]

"También probamos el componente `Button.vue` montándolo con `@vue/test-utils`: que renderiza el label, que respeta los props de tipo y disabled, que inyecta los colores como variables CSS, y que emite el evento click.

Y aquí está lo más valioso de toda la tarea: **este test encontró un bug real el primer día**. Esperábamos una emisión del evento click, pero recibimos dos."

[PANTALLA: VS Code — abrir `frontend/src/components/UI/Button/Button.vue` , señalar la línea `emits: ['click']`]

"La causa: el componente no declaraba sus eventos en la opción `emits` , así que Vue registraba el click nativo del DOM **además** del `$emit` manual — cualquier componente padre recibía el click duplicado. Lo corregimos declarando `emits: ['click']` , y el test quedó como **prueba de regresión** que valida exactamente una emisión.

Por último, probamos los helpers de estado y formato. Un detalle técnico: para probar `isOverdue` , que compara contra la fecha actual, usamos los **fake timers** de Vitest para congelar el reloj — así el test da siempre el mismo resultado, hoy y dentro de un año."

[PANTALLA: terminal en `frontend/`]

Ejecutar en vivo:

```
npm test
```

"Corriendo en vivo: **31 pruebas, todas en verde, en menos de un segundo**. Y ahora la parte de automatización que más nos gustó: el modo watch."

Ejecutar en vivo:

```
npm run test:watch
```

Con watch corriendo, abrir `frontend/src/utils/statusHelpers.js` y romper algo a propósito — por ejemplo, en `formatMoney` cambiar `n >= 1_000_000` por `n >= 2_000_000`. Guardar.

"Vitest está vigilando los archivos. Voy a introducir un error a propósito en `formatMoney` ... guardo... y **de forma instantánea** solo se re-ejecutaron los tests afectados, y el fallo nos muestra un diff clarísimo: esperaba '2.5M' y recibió otra cosa."

Deshacer el cambio (Ctrl+Z), guardar.

"Deshago el cambio, guardo... y todo verde otra vez. Este ciclo de feedback de menos de un segundo es lo que hace que realmente corras las pruebas mientras programas, no solo al final. Salgo del modo watch con la tecla q."

ESCENA 6 — Resultados y conclusiones · PERSONA 1 (~1.5 minutos) — *rubro de 20 pts*

[PANTALLA: diapositiva 10 — resultados]

"Para cerrar, los números: escribimos **53 pruebas unitarias** — 22 de backend y 31 de frontend — que corren en **menos de un segundo por suite**, con alrededor de **80 % de cobertura** en los archivos probados, y que ya nos sirvieron para **detectar y corregir un bug real**."

En tiempo invertido: la configuración de Vitest en los dos paquetes nos tomó unos 15 minutos, y escribir las ocho suites alrededor de una hora y media. En total, **menos de dos horas** para dejar protegidas las rutas más críticas del sistema: autenticación, autorización, validación y lógica de negocio."

[PANTALLA: diapositiva 11 — conclusiones]

"Nuestras conclusiones:

Primera: la **configuración fue casi nula** — 8 líneas en el backend, y el frontend heredó la configuración de Vite completa.

Segunda: la **velocidad cambia la forma de trabajar**. Con suites de menos de un segundo y watch instantáneo, correr los tests en cada guardado es viable — no es algo que dejas solo para el CI.

Tercera: el **mocking integrado nos alcanzó para todo**: vi.fn para simular Express, stubGlobal para fetch, y fake timers para las fechas, sin instalar ninguna librería adicional.

Cuarta: los tests **demonstraron su valor el primer día** encontrando el bug del evento duplicado en Button.

Y como contras honestos: la comunidad de Vitest es más joven que la de Jest — hay menos respuestas históricas en Stack Overflow — y su mayor ventaja depende de usar Vite; en un proyecto sin Vite la diferencia se reduce.

Nuestro veredicto: **Vitest fue la elección correcta para Kontrol** — en dos horas obtuvimos 53 pruebas listas para integrarse a CI. Gracias por su atención."

Notas finales de grabación

- Si un comando tarda en arrancar la primera vez (npm cachea después), corran cada comando una vez antes de grabar.
- Si se equivocan al hablar, pausen y repitan la frase — es más fácil cortar en edición que regrabar la escena.
- Verificar al final que en el video se vea con claridad: el código de al menos 3 tests de

backend y 3 de frontend, la ejecución en verde de ambas suites, el reporte de cobertura y el modo watch fallando y recuperándose. Eso es exactamente lo que pide el rubro de 35 puntos.